



Proyecto Final

Presentado por:

JHOJAN SERNA HENAO
Código: 2259504

JUAN CAMILO GARCÍA
Código: 2259416

JEAN CARLOS LERMA
Código: —

Docente:

Luz Carime Lucumí Hernández

Procesamiento de Lenguaje Natural
Escuela de Ingeniería de Sistemas y Computación
Programa Académico de Ingeniería de Sistemas
Universidad del Valle
2026–1

1. Definición Formal

1.1. Autómata implementado

Este proyecto no usa un autómata finito convencional de 5-tuplas $(Q, \Sigma, \delta, q_0, F)$. El lenguaje de las recetas es un **Lenguaje Libre de Contexto**. Los autómatas finitos solo reconocen lenguajes regulares y no pueden manejar la anidación jerárquica que aquí se necesita.

En cambio, el modelo equivalente es un **Autómata de Pila** (*Pushdown Automaton*). En el código, esto se implementa con un *Parser* de Búsqueda Best-First (A^*), que maneja una cola de prioridad con estados de la forma (posicion, pendientes, pila_de_nodos).

1.2. Gramática formal: $G = (V, T, P, S)$

La gramática se define como la 4-tupla $G = (V, T, P, S)$:

- S (**Símbolo Inicial**): S
- V (**Variables / No Terminales**):
 $\{S, LISTA_INS, INS, VP, COMPS, COMP, NP, PP, PARAM, V, N, ADJ, DET, QUANT, PREP, LEVEL, NUM, ORDEN\}$
- T (**Terminales**): El token especial $\langle EOS \rangle$, palabras literales (durante, minutos, mas, a, fuego, al, final, en, aceite, caliente) y todas las palabras del léxico base (LEXICO_RASGOS).
- P (**Producciones**): Ver bloque a continuación.

Listing 1: Producciones de la gramática

```
# Estructura de nivel superior
S      -> LISTA_INS
LISTA_INS -> INS <EOS> | INS <EOS> LISTA_INS
INS     -> VP | ORDEN VP
VP      -> V COMPS | V
COMPS   -> COMP COMPS | COMP
COMP    -> NP | PP | PARAM

# Sintagmas
NP  -> DET N ADJ | DET N | N ADJ | QUANT N | N
PP  -> PREP NP | PREP NUM N | PREP DET N | PREP DET N ADJ

# Parametros especiales
PARAM -> 'durante' NUM 'minutos'
        | 'durante' NUM 'minutos' 'mas'
        | 'a' 'fuego' LEVEL
        | 'al' 'final'
        | 'en' 'aceite' 'caliente'

# Producciones lexicas (generadas desde el diccionario de rasgos)
V      -> 'corta' | 'agrega' | 'cocina' | 'mezcla' | ...
N      -> 'gallina' | 'sal' | 'agua' | 'cebolla' | ...
```

```

ADJ    -> 'cortada' | 'grande' | 'picada' | ...
DET    -> 'la' | 'el' | 'un' | 'una' | ...
PREP   -> 'en' | 'con' | ...
NUM    -> 'cuarenta' | 'uno' | 'dos' | ...
LEVEL  -> 'medio' | 'alto' | 'bajo'
ORDEN  -> 'primero' | 'despues'
QUANT  -> 'suficiente' | 'al gusto' | ...

```

2. Estrategias Técnicas

2.1. Tokenización con marcadores EOS

Cada línea de la receta se convierte en una lista de tokens normalizada. Al final de cada instrucción se agrega el token <EOS>. La gramática usa ese token para saber dónde termina cada instrucción.

Ejemplo:

```

"Agrega la Sal\nCocina a fuego medio" ->
['agrega', 'la', 'sal', '<EOS>', 'cocina', 'a', 'fuego', '
  medio', '<EOS>']

```

2.2. Gramática libre de contexto (CFG)

Las reglas de producción definen la estructura de cada instrucción. Las reglas léxicas (V, N, ADJ...) se generan desde el léxico. Agregar una palabra nueva al léxico la incorpora sola a la gramática.

2.3. DGS — Unificación de rasgos lingüísticos

Cada palabra tiene rasgos de categoría, género y número. Cuando el parser arma un NP, aplica unificación: combina los rasgos de los componentes. Si hay conflicto, descarta esa derivación. Así se valida la concordancia sin escribir una regla por cada combinación posible.

La tabla 1 muestra casos de aceptación y rechazo.

Table 1: Ejemplos de unificación de rasgos

Frase	DET	N	ADJ	Resultado
la gallina cortada	fem, sing	fem, sing	fem, sing	válido ✓
el pollo molido	masc, sing	masc, sing	masc, sing	válido ✓
el gallina	masc, sing	fem, sing	—	rechazado ×
los pollo	masc, plur	masc, sing	—	rechazado ×

2.4. PCFG — Resolución de ambigüedad con probabilidades

Cuando la gramática acepta varias derivaciones para la misma secuencia, la PCFG asigna una probabilidad a cada regla. El parser escoge la derivación más probable.

Listing 2: grammar/probabilidades.py

```

1  PROB_TABLE = {
2      # "a fuego medio" se prefiere como PARAM, no como PP
3      ("COMP", ("NP",)): 0.10,
4      ("COMP", ("PP",)): 0.30,
5      ("COMP", ("PARAM",)): 0.60,    # mayor peso
6
7      # Se prefieren NPs mas completos si la concordancia es
        valida
8      ("NP", ("DET", "N", "ADJ")): 0.35,
9      ("NP", ("DET", "N")): 0.25,
10     ("NP", ("N", "ADJ")): 0.20,
11     ("NP", ("QUANT", "N")): 0.10,
12     ("NP", ("N",)): 0.10,
13
14     ("VP", ("V", "COMPS")): 0.70,
15     ("VP", ("V",)): 0.30,
16 }

```

El algoritmo es **Best-First** con cola de prioridad (heapq). El costo de cada estado es $-\log(\text{prob acumulada})$, lo que convierte el problema en una búsqueda de costo mínimo equivalente a Dijkstra / A*.

Listing 3: core/pcfg_bestfirst.py — nucleo del algoritmo

```

1  pq = [] # min-heap: (costo, tie_breaker, estado)
2  heapq.heappush(pq, (0.0, 0, State(pos=0, pending=('S',),
        stack=())))
3
4  while pq:
5      cost, _, st = heapq.heappop(pq) # expandir el mas
        probable
6      sym = st.pending[0]
7      for rule in pcfg.rules(sym): # expandir no terminal
8          new_cost = cost + (-rule.logp) # acumular -log(prob)
9          heapq.heappush(pq, (new_cost, ..., next_state))

```